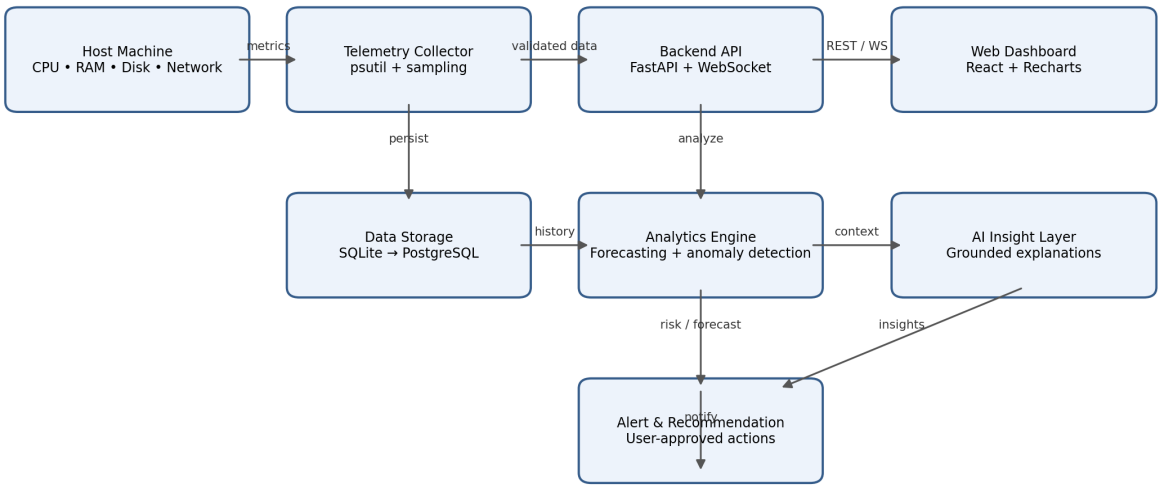
SysPredictAI

AI-Powered Real-Time System Monitoring and Predictive Analytics Platform

Project Documentation | Version 1.0 | 12 November 2026

SysPredictAI — High-Level Architecture



A practical technical blueprint for an incremental student-built implementation using Python, FastAPI, machine learning, and a React dashboard.

1. Project Overview

SysPredictAI is a system observability and predictive analytics application. It collects host performance telemetry, presents live system health, analyzes historical patterns, identifies unusual behavior, and estimates short-term resource utilization. It is designed to help a user understand what is happening on a machine and act before resource pressure becomes severe.

Problem statement. Basic task managers show current resource usage but provide limited historical analysis, early warning, or contextual explanations. SysPredictAI combines continuous telemetry, trend analysis, anomaly detection, and actionable recommendations in one dashboard.

Primary objectives

- Collect CPU, memory, disk, network, and process-level metrics at a configurable interval.
- Expose current and historical metrics through a documented backend API.
- Visualize live readings and trends in a responsive dashboard.
- Forecast near-term CPU and memory utilization when enough valid history exists.
- Detect unusual metric patterns and generate explainable alerts.
- Provide recommendations grounded in observed telemetry; require user approval for system-changing actions.

Scope boundary. The initial project predicts resource utilization and detects telemetry anomalies. It does not claim to predict arbitrary hardware failures unless representative labeled failure data is collected and validated.

2. Users and Use Cases

- Student/developer: monitor their development machine while running applications.
- IT support/operator: inspect resource pressure and identify processes consuming resources.
- Project evaluator: review historical graphs, anomaly events, forecasts, and model evaluation.

3. Functional Requirements

- FR-01: Collect CPU percentage, memory usage, disk usage, network throughput, and process statistics.
- FR-02: Store timestamped telemetry records and expose recent history.
- FR-03: Provide REST endpoints for health, current metrics, history, processes, forecasts, and alerts.
- FR-04: Stream live metric updates to connected dashboard clients using WebSockets.
- FR-05: Generate CPU/RAM forecasts only when sufficient valid history is available.
- FR-06: Detect anomalies and retain event timestamp, metric, observed value, severity, and reason.
- FR-07: Display charts, process information, alert history, and system status.
- FR-08: Generate plain-language explanations from actual metric and alert context.
- FR-09: Allow configuration of sampling interval and alert thresholds.

4. Non-Functional Requirements

- Reliability: collector and API failures should be logged and recoverable without silently fabricating metrics.
- Performance: use asynchronous streaming and bounded history queries; measure latency rather than promising an unverified response time.
- Security: validate inputs, keep API keys in environment variables, and avoid exposing process details beyond the local user context.

- Maintainability: separate collection, API, storage, analytics, and UI modules.
- Usability: dashboard labels should include units, timestamps, and clear loading/error states.
- Portability: start with local development; document OS-specific metric limitations.

5. System Architecture

The system follows a layered architecture: telemetry collection, normalization, API and persistence, analytics, alerting, and presentation. The browser receives live readings through WebSockets and requests historical or analytical data through REST endpoints.

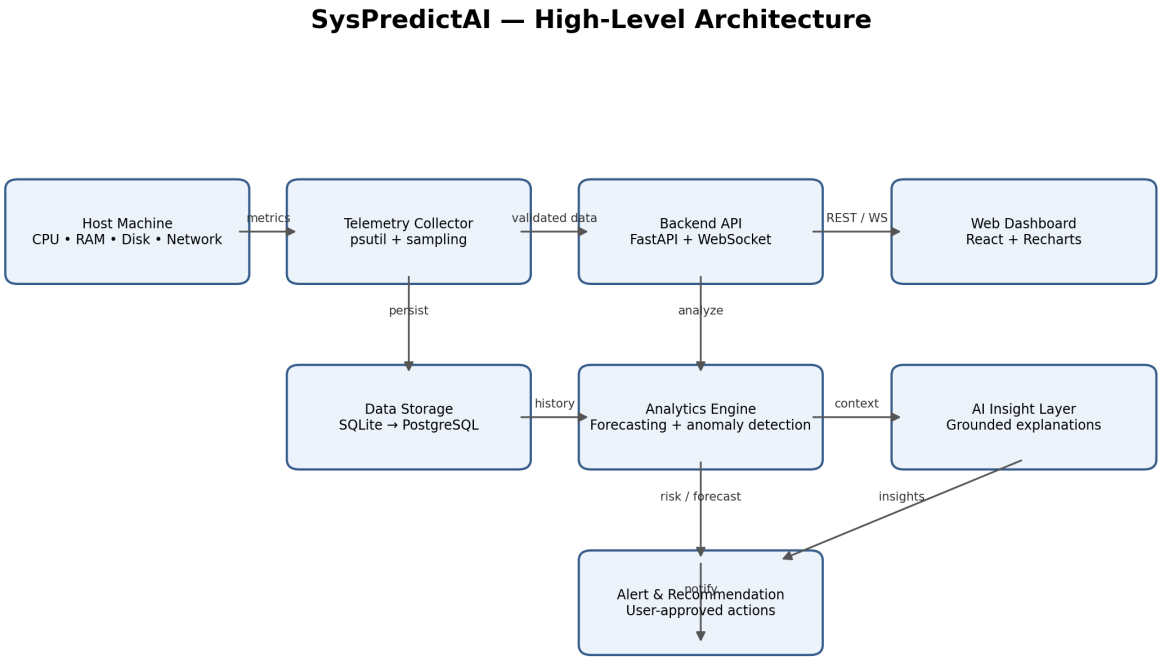


Figure 1. High-Level Design (HLD)

6. Low-Level Design

The collector samples the host using psutil. Raw values are normalized into typed schemas, validated, timestamped, and sent to the persistence layer. The analytics pipeline reads historical samples, constructs time-window features, runs forecasting and anomaly models, and emits structured results to the API and alert engine.

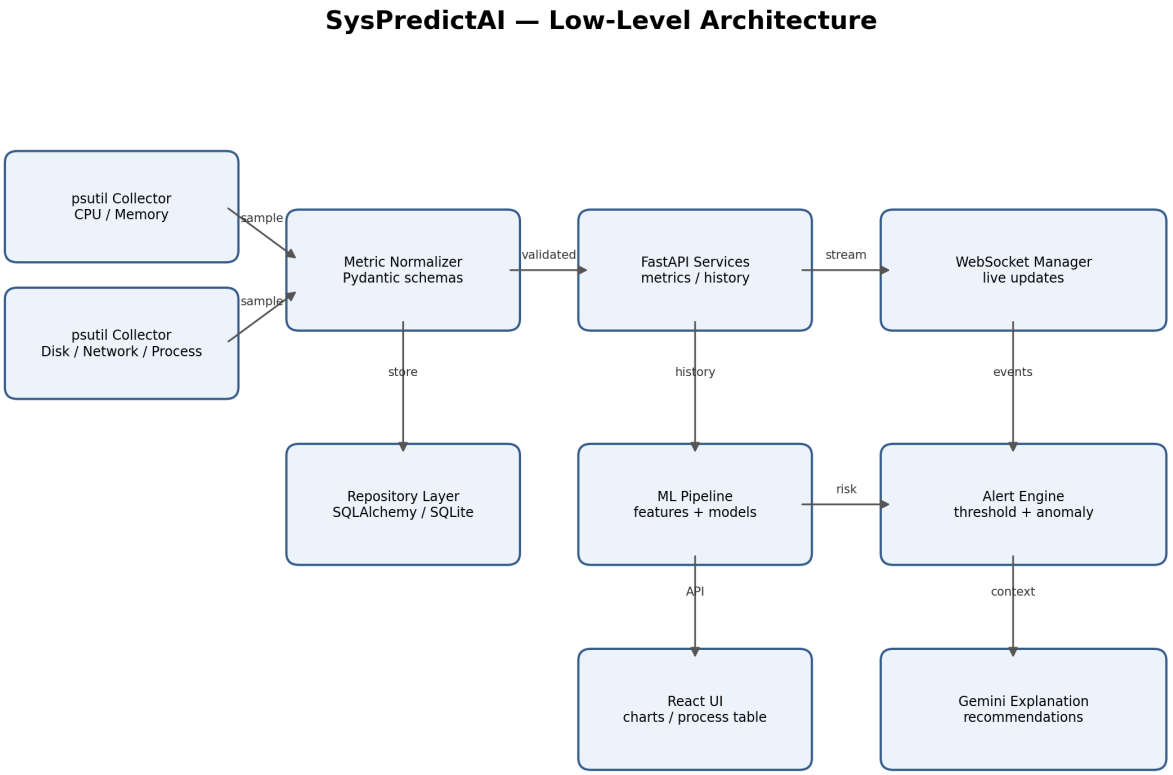


Figure 2. Low-Level Design (LLD)

Suggested module structure

```
backend/
app/main.py
app/api/routes_metrics.py
app/api/routes_alerts.py
app/collectors/system_collector.py
app/schemas/metrics.py
app/services/metrics_service.py
app/services/alert_service.py
app/analytics/features.py
app/analytics/forecast.py
app/analytics/anomaly.py
app/db/models.py
app/db/repository.py
tests/
frontend/src/
```

7. Data Design

Core tables / entities

Entity	Important fields	Purpose
MetricSample	id, timestamp, cpu_pct, memory_pct, disk_pct, bytes_sent, bytes_recv	Time-series host telemetry

Entity	Important fields	Purpose
ProcessSample	id, timestamp, pid, name, cpu_pct, memory_pct	Process-level snapshots
Forecast	id, created_at, metric_name, horizon, predicted_value, model_version	Stored model outputs
AnomalyEvent	id, timestamp, metric_name, observed_value, score, severity, explanation	Detected unusual behavior
Alert	id, created_at, severity, title, message, acknowledged_at	User-facing notifications
Settings	key, value, updated_at	Sampling and threshold configuration

Use SQLite for the first local prototype. Keep repository interfaces database-agnostic so PostgreSQL can be introduced later without rewriting business logic.

8. API Design

Method	Endpoint	Description
GET	/api/health	Service health and version
GET	/api/metrics/current	Latest host metrics
GET	/api/metrics/history?range=1h	Historical samples for a bounded range
GET	/api/processes	Current top processes
GET	/api/forecasts	Latest available forecasts
GET	/api/alerts	Recent alerts and anomaly events
WS	/ws/metrics	Live metric stream

Example metric response (illustrative schema only; runtime values must come from the collector):

```
{ "timestamp": "ISO-8601", "cpu_percent": 24.5,
"memory_percent": 61.2, "disk_percent": 72.0,
"network": {"bytes_sent": 12345, "bytes_recv": 67890} }
```

9. Machine Learning and Analytics

Forecasting: begin with a simple baseline (last-value or moving average). After collecting enough timestamped samples, train and evaluate a regression model such as Random Forest using lag features and rolling statistics. Use chronological train/test splits; never randomly shuffle time-series samples. Report MAE or RMSE and compare against the baseline.

Anomaly detection: start with transparent threshold/rule checks for obvious conditions. Then evaluate Isolation Forest or another unsupervised method on normalized features. Persist anomaly score and supporting metric values. Tune thresholds against test scenarios to reduce noisy alerts.

Feature examples: current value, 1/5/15-minute rolling mean, rolling standard deviation, recent slope, lagged values, and time since last sample.

Cold start: when there is not enough history, display “insufficient history” or a baseline estimate rather than fabricating a model prediction.

10. AI Explanation Layer

An LLM can convert structured telemetry and alert records into concise explanations and recommended next checks. It should not invent root causes. Provide only the relevant metric window, anomaly details, and system context; ask the model to distinguish observed facts from hypotheses. Keep API credentials server-side. Any remediation that changes system state must require explicit user confirmation.

11. User Interface Design

- Overview: current CPU, RAM, disk, and network cards with last-updated timestamp.
- Live monitoring: rolling line charts and connection status.
- Processes: sortable table of process name, PID, CPU, and memory.
- Predictions: forecast chart, horizon, model version, and evaluation metric.
- Anomalies & alerts: severity, time, metric, observed value, and explanation.
- AI insights: evidence-based explanation and safe recommended next steps.
- Settings: sampling interval, alert thresholds, and retention period.

12. Technology Stack

Layer	Technology	Role
Telemetry	Python, psutil	Collect host metrics
Backend	FastAPI, Pydantic, WebSockets	REST API, validation, live updates
Storage	SQLite initially; PostgreSQL later	Persist time-series and events
Data processing	Pandas, NumPy	Feature engineering and analysis
Machine learning	Scikit-learn; optional XGBoost	Forecasting and anomaly detection
Frontend	React, Tailwind CSS, Recharts	Responsive dashboard and charts
AI explanation	Gemini API (optional)	Grounded natural-language insights
Development	Git, GitHub, Antigravity, AI Studio	Version control and assisted development

13. Four-Day Development Plan

Day 1 — Monitoring backend

- Create project structure and virtual environment; install FastAPI, Uvicorn, psutil, SQLAlchemy, and pytest.
- Implement CPU, memory, disk, network, and process collectors.
- Create Pydantic response schemas, health/current/history/process endpoints, and SQLite persistence.
- Add a WebSocket endpoint for periodic live metrics; write basic collector and API tests.

Day 2 — Predictive analytics

- Create a repeatable telemetry dataset and feature-generation pipeline.
- Implement baseline forecasts, then a trained model if enough history is available.
- Add anomaly rules and evaluate Isolation Forest on controlled scenarios.
- Record evaluation results and expose forecast/anomaly endpoints.

Day 3 — Frontend and explanations

- Build the React dashboard with metric cards, live charts, process table, and alert panel.
- Connect REST and WebSocket flows; add loading, empty, disconnected, and error states.
- Add optional AI explanations using only supplied telemetry and anomaly evidence.

Day 4 — Integration and validation

- Run end-to-end tests and controlled CPU/memory/disk/network activity tests.
- Verify persistence, reconnect behavior, alert quality, and forecast evaluation.
- Fix defects, document setup, add screenshots, and prepare a reproducible demo.

14. Testing and Acceptance Criteria

- Collector tests: valid ranges, timestamps, unavailable metric handling, and no random placeholder values.
- API tests: expected status codes, response schemas, invalid query handling, and bounded history.
- WebSocket tests: client receives timestamped updates and reconnects cleanly.
- ML tests: chronological split, baseline comparison, metrics recorded, and insufficient-history behavior.
- Anomaly tests: known synthetic spikes are detected; normal periods do not trigger excessive alerts.
- UI tests: charts render real API data; empty/error/disconnected states are visible.
- Security checks: secrets are not committed; user-controlled inputs are validated.

Acceptance is based on reproducible test results and observed application behavior, not on features listed in this design document.

15. Risks and Mitigations

- OS differences: document platform-specific metrics and gracefully handle unsupported counters.
- No historical data: use baseline/insufficient-history states until enough samples exist.
- False alerts: tune thresholds and evaluate on controlled normal and stressed periods.
- LLM hallucination: constrain prompts to telemetry evidence and label hypotheses.
- Performance overhead: use configurable sampling and avoid expensive per-process polling too frequently.
- Security: keep the dashboard local by default; add authentication and authorization before remote exposure.

16. Setup and Run (Development Baseline)

Example backend setup on a machine with Python installed:

```
python -m venv .venv
# Windows PowerShell: .venv\\Scripts\\Activate.ps1
# macOS/Linux: source .venv/bin/activate
pip install fastapi uvicorn psutil sqlalchemy pydantic pytest
uvicorn app.main:app --reload
```

The exact command depends on the final folder structure and entry-point module. Add a .env file for configuration, keep it out of Git, and commit a .env.example containing placeholder variable names only.

17. Future Enhancements

- Longer-horizon forecasting and per-process trend analysis.
- Configurable dashboards and alert delivery integrations.
- Container and multi-machine monitoring.
- Role-based access and secure remote deployment.
- Model drift monitoring and scheduled retraining using validated data.

18. Conclusion

SysPredictAI is designed as a practical end-to-end system monitoring project that combines real telemetry, reliable APIs, historical storage, predictive analytics, anomaly detection, and an evidence-grounded dashboard. Build and verify each layer incrementally. Treat this document as the design baseline and update implementation details, screenshots, and measured results as the project is completed.

Documentation note: This PDF describes the proposed design and development plan. It does not certify that the application, models, integrations, or performance targets have already been implemented or tested.